

UNIVERSIDAD RAFAEL LANDIVAR
FACULTAD DE INGENIERIA
LENGUAJES FORMALES Y AUTOMATAS
ING. ROJAS ARGUETA CESAR SALVADOR

Documento de análisis y diseño
Proyecto 2

INTEGRANTES:
RODRIGO RUIZ 1037623
JAROD BOLAÑOS 1251621

CIUDAD DE GUATEMALA

Indice

Propósito	3
Descripción general de la solución	3
Glosario de términos y clases y estructura de los datos	3
Mapa por opción del menú	5
Expresiones regulares	7
Algoritmo de construcción de subconjuntos	8
Aclaraciones de variables y funcionamiento	8
automata.py	8
cargador.py	9
cargador_afnd.py	9
validador.py	10
validador_afnd.py	10
conversor.py	11
simulador.py	11
colores.py	12
main.py	12
Restricciones	13
Restricciones generales del proyecto	13
Reglas de acceso entre opciones del menú	13
Diagramas de flujo del proyecto	14
Diagrama principal	14
Diagrama Crear un AFD manualmente	15
Diagrama Cargar un AFD desde archivo .txt	16
Diagrama Crear un AFND manualmente	17
Diagrama Cargar un AFND desde archivo .txt	18
Diagrama Mostrar la definición formal y la tabla del autómata cargado	19
Diagrama Validar la estructura del autómata	20
Diagrama Convertir el AFND cargado en un AFD equivalente	22
Diagrama Mostrar la tabla de equivalencias de macroestados	23
Diagrama Mostrar la tabla de transición del AFD generado	24
Diagrama Evaluar una cadena	25
Diagrama Evaluar un archivo de cadenas	26
Diagrama Consultar el historial de evaluaciones	27
Diagrama Realizar el análisis estructural	28
Diagrama Cargar o crear otro autómata	29
Diagrama Salir	30
Casos de prueba	30

Propósito

El propósito de este programa es realizar un programa que sea capaz de obtener los datos de un AFND y un AFN al mismo tiempo que lograr obtener sus datos validar si es viable o no y si es posible ejemplificar como seria el recorrido de ese diagrama junto con sus transiciones y forma ampliando lo entregado en la Fase 1 del proyecto y los objetivos de este.

Descripción general de la solución

La solución trata de 15 casos concretos que se trata de evolucionar los 10 casos anteriores en este caso se abarcan los siguientes casos

1. Crear un AFD manualmente.
2. Cargar un AFD desde un archivo .txt.
3. Crear un AFND manualmente.
4. Cargar un AFND desde un archivo .txt.
5. Mostrar la definición formal y la tabla del autómata cargado.
6. Validar la estructura del autómata.
7. Convertir el AFND cargado en un AFD equivalente.
8. Mostrar la tabla de equivalencias de macroestados.
9. Mostrar la tabla de transición del AFD generado.
10. Evaluar una cadena.
11. Evaluar un archivo de cadenas.
12. Consultar el historial de evaluaciones.
13. Realizar el análisis estructural.
14. Cargar o crear otro autómata.
15. Salir.

En este caso se explicarán cuáles son las partes que conforman cada uno de los casos y como funcionan para hacerlo un poco mas estable primero se explicara que es cada una de las clases, como funciona y que es lo que hace.

Glosario de términos y clases y estructura de los datos

Término	Dónde vive	Qué es
AFD	automata.py	Quíntupla $M=(Q,\Sigma,\delta,q_0,F)$. delta: dict[(estado,simbolo)] -> str (un solo destino, o ausente si está incompleto).
AFND	automata.py	Misma quíntupla, pero delta: dict[(estado,simbolo)] ->

		frozenset[str] (0, 1 o varios destinos).
Macroestado	Concepto, no una clase	Un subconjunto (frozenset) de estados del AFND. Cada macroestado alcanzado durante la conversión se convierte en UN estado del AFD generado.
Etiqueta	conversor._siguiente_etiqueta	Nombre legible que se le asigna a un macroestado, en el orden en que se descubre: A, B, C, ..., Z, AA, AB, ...
tabla_equivalencias	Variable de sesión en main.py; la arma conversor.construir_afd_desde_afnd	dict[etiqueta] -> frozenset (el macroestado que esa etiqueta representa). Es lo que muestra la opción 8.
automata_cargado	Variable de sesión en main.main()	El AFD o AFND recién creado/cargado con las opciones 1-4. Se distingue con isinstance.
afd_generado	Variable de sesión en main.main()	El AFD que resulta de convertir automata_cargado (opción 7). None mientras no se convierte.
historial	Instancia de Historial (clase en main.py)	Registro de cadenas evaluadas contra el autómata activo. Se reinicia en las opciones 1-4 y 14.
ResultadoEvaluacion / Paso	Dataclasses en simulador.py	Resultado de evaluar una cadena (cadena, traza, veredicto, motivo) y cada paso de la traza (estado_origen, simbolo, estado_destino).
AnalisisEstructural	Dataclass en validador.py	alcanzables, inaccesibles, finales_alcanzables, lenguaje_vacio: resultado del análisis estructural (opción 13, y automático tras validar un AFD).
obtener_afd_operable	Función en main.py	Devuelve el AFD para las opciones 10, 11 y 13: automata_cargado si ya es un AFD validado, o afd_generado si automata_cargado es un AFND ya convertido.

Mapa por opción del menú

Opción	Módulo / función	Clases	Datos de entrada	Restricciones clave
1. Crear un AFD manualmente	cargador.crear_afd_manual() (usa _pedir_conjunto, _pedir_conjunto_final, _pedir_transiciones)	AFD	Nombre; Q; Σ ; q0; F (puede ir vacía); transiciones origen,símbolo,destino hasta FIN	Nombre no vacío; Q/ Σ sin vacíos ni duplicados; Σ sin épsilon; q0 $\in$ Q; F $\subseteq$ Q; origen/destino $\in$ Q, símbolo $\in$ Σ ; (origen,símbolo) sin repetir ("sería un AFND")
2. Cargar un AFD desde .txt	cargador.cargar_afd_desde_archivo(ruta) (+ normalizar_ruta_archivo)	AFD	Ruta del archivo	Cada línea debe calzar con un PATRON_*; épsilon prohibido; (origen,símbolo) repetido = error; deben existir NOMBRE, ESTADOS, ALFABETO, INICIAL
3. Crear un AFND manualmente	cargador_afnd.crear_afnd_manual() (+ _pedir_transiciones_afnd; reutiliza funciones de cargador.py)	AFND	Igual que la 1, pero transición origen,símbolo,destino 1 destino2 ... ($\emptyset$ o vacío = sin destino)	Igual que la 1, salvo que un (origen,símbolo) admite varios destinos si se declaran juntos con ; declararlo dos veces por separado es error
4. Cargar un AFND desde .txt	cargador_afnd.cargar_afnd_desde_archivo(ruta)	AFND	Ruta del archivo	Regex PATRON_TRANSICION_AFND; TIPO= opcional (aviso si no dice AFND); $\emptyset$ /vacío = sin destino; épsilon prohibido ("no forma parte del alcance de esta fase")

5. Mostrar definición formal y tabla	main.accion_mostrar_automata_cargado despacha por isinstance a funciones de AFD o AFND	AFD o AFND	Ninguno (usa automata_cargado)	Requiere automata_cargado is not None
6. Validar la estructura	main.accion_validar_automata_cargado accion_validar_afd o accion_validar_afnd	AFD/AFND, AnalisisEstructural	Solo AFD incompleto: S/N para agregar estado de trampa	AFD: $q_0 \in Q$, $F \subseteq Q$, transiciones consistentes, completitud (o trampa). AFND: $q_0 \in Q$, $F \subseteq Q$, $destinos \subseteq Q$; no existe "incompleto"
7. Convertir AFND en AFD	main.accion_convertir conversor.construir_afd_de sde_afnd	AFND (entrada), AFD (salida), tabla_equivalencias	Ninguno	automata_cargado debe ser AFND y estar validado; construcción de subconjuntos
8. Tabla de equivalencias	main.accion_mostrar_equiv_alencias	—	Ninguno	Requiere tabla_equivalencias (opción 7 exitosa)
9. Tabla del AFD generado	main.accion_mostrar_tabla(afd_generado) (la misma de la opción 5)	AFD	Ninguno	Requiere afd_generado is not None
10. Evaluar una cadena	main.accion_evaluar_cadena simulador.evaluar_cadena, simulador.imprimir_traza	Resultado Evaluación, Paso, Historial	Cadena (vacía = λ)	Requiere obtener_afd_operable(...) is not None
11. Evaluar un archivo de cadenas	main.accion_evaluar_archivo simulador.evaluar_lote	Resultado Evaluación, Historial	Ruta del archivo (una cadena por línea)	Igual que la 10, aplicado a cada línea

12. Consultar historial	Historial.mostrar()	Historial, RegistroHistorial	Ninguno	Si está vacío solo informa; se reinicia con 1-4 y 14
13. Análisis estructural	main.imprimir_analisis_estructural validador.analizar_estructura	AnalisisEstructural	Ninguno	Misma guardia que 10/11 (obtener_afd_operable)
14. Cargar o crear otro autómata	Bucle de main(), rama opcion == 14	—	Ninguno	Reinicia automata_cargado, afd_generado, tabla_equivalencias e historial
15. Salir	Bucle de main(), break	—		

Expresiones regulares

Patrón	Módulo	Qué reconoce
PATRON_NOMBRE	cargador.py / cargador_afnd.py	Línea NOMBRE=...
PATRON_TIPO	cargador_afnd.py	Línea opcional TIPO=... (aviso si no dice AFND)
PATRON_ESTADOS	Ambos	Línea ESTADOS=...
PATRON_ALFABETO	Ambos	Línea ALFABETO=...
PATRON_INICIAL	Ambos	Línea INICIAL=...
PATRON_FINALES	Ambos	Línea FINALES=... (puede ir vacía)
PATRON_ENCABEZADO_TRANSICIONES	Ambos	La línea literal TRANSICIONES:
PATRON_TRANSICION	cargador.py	origen,simbolo,destino (un solo destino, AFD)
PATRON_TRANSICION_AFND	cargador_afnd.py	origen,simbolo,destino...: el destino es el resto de la línea, luego se separa por
PATRON_IDENTIFICADOR	cargador.py (importado por cargador_afnd.py)	Nombre de estado válido: [A-Za-z0-9_]+

Algoritmo de construcción de subconjuntos

Paso	Qué hace	Línea / idea clave del código
1	Macroestado inicial	frozenset({afnd.q0})
2	Cola de pendientes	deque([macroestado_inicial]), procesada con popleft() (BFS)
3	Para cada macroestado y cada símbolo	destino = unión de afnd.delta.get((estado,simbolo), frozenset()) para cada estado del macroestado
4	¿Macroestado nuevo?	Si destino no está en etiquetas, se le asigna la siguiente etiqueta y se encola
5	Se registra la transición del AFD	delta_afd[(etiqueta_actual, simbolo)] = etiqueta_destino
6	Termina cuando la cola queda vacía	Ya se descubrieron todos los macroestados alcanzables
7	Estados finales del AFD	Un macroestado es final si macroestado & afnd.F no es vacío
8	Resultado	Un AFD (validado = True, completo y determinista por construcción) + tabla_equivalecias

Aclaraciones de variables y funcionamiento

automata.py

La clase AFD (líneas 23-71) se mantiene sin cambios respecto a la Fase 1. Su constructor (`__init__`, líneas 50-57) construye la quintupla del autómata, y la variable central es `self.delta = dict(delta)` (línea 54), un diccionario que asocia cada par (estado, símbolo) con un único destino. En la línea 57 se establece `self.validado = False`, de modo que todo autómata nace sin validar. El método `transicion` (líneas 59-65) implementa la función de transición δ de forma genérica mediante `delta.get((estado, simbolo))`, lo que evita cualquier cadena de `if/elif`; si no existe la transición devuelve `None`, lo que indica que el AFD está incompleto.

La clase AFND (líneas 74-127) es nueva en la Fase 2 y sigue el mismo patrón que AFD, con una diferencia clave. En la línea 108, `self.delta = {clave: frozenset(destinos) for clave, destinos in delta.items()}` define la estructura de datos más importante de toda la fase: cada valor del diccionario de transiciones es un `frozenset` y no un `str`, lo que permite representar cero, uno o varios destinos. El método `transicion` (líneas 113-121) devuelve `self.delta.get((estado, simbolo), frozenset())`. El segundo argumento de `.get` marca la diferencia con el AFD: mientras en este la ausencia de una transición devuelve `None` y se interpreta como un error de completitud, en el AFND devuelve el conjunto vacío ($\emptyset$), que es un resultado válido. Esta única línea explica por qué un AFND no tiene transiciones "incompletas".

cargador.py

Este módulo no tiene cambios de código respecto a la Fase 1. Entre las líneas 38 y 48 se definen las expresiones regulares (PATRON_NOMBRE hasta PATRON_TRANSICION, además de PATRON_IDENTIFICADOR), y en la línea 52 el conjunto SIMBOLOS_PROHIBIDOS. La función crear_afd_manual (líneas 68-96) y sus funciones auxiliares _pedir_conjunto (99-114), _pedir_conjunto_final (117-132) y _pedir_transiciones (135-173) siguen un patrón de entrada con ciclo while: cada una solicita un dato, lo valida (sin vacíos, sin duplicados, sin épsilon y, cuando corresponde, como subconjunto de Q) y vuelve a pedirlo si la validación falla.

La función normalizar_ruta_archivo (líneas 180-201) realiza una acción de saneamiento y no de validación: elimina comillas, expande ~ y variables de entorno y unifica los separadores de ruta. Recibe como dato de entrada una ruta en forma de cadena y no aplica ninguna restricción estructural. Por su parte, cargar_afd_desde_archivo (líneas 204-334) conserva el comportamiento documentado en la Fase 1.

En la Fase 2, estas funciones son invocadas por main.accion_crear_manual() y main.accion_cargar_archivo(), y su resultado ya no se almacena en una variable afd fija, sino en automata_cargado, que también puede contener un AFND.

cargador_afnd.py

Entre las líneas 16 y 23, este módulo importa desde cargador.py los elementos PATRON_IDENTIFICADOR, SIMBOLOS_PROHIBIDOS, _pedir_conjunto, _pedir_conjunto_final, _separar_lista y normalizar_ruta_archivo, en lugar de duplicarlos, lo que constituye un ejemplo directo de reutilización de código.

Las expresiones regulares nuevas se encuentran entre las líneas 32 y 40. PATRON_TIPO (línea 33) es la única que no existe en cargador.py. En PATRON_TRANSICION_AFND (línea 40), el tercer grupo es (.*)\$, que acepta cualquier contenido, en lugar de ([A-Za-z0-9_]+)\$ como en el AFD; ahí reside la diferencia entre aceptar un único destino y aceptar varios destinos separados por |. En la línea 44, TOKENS_SIN_DESTINO = {'Ø', ''} establece la convención de qué se considera "sin destino".

La función _separar_destinos (líneas 47-56) realiza el cálculo que transforma el texto leído en el tipo de dato que exige AFND.delta: si el texto pertenece a TOKENS_SIN_DESTINO devuelve frozenset() (línea 54); de lo contrario, separa por | y construye el frozenset correspondiente (línea 56).

La función crear_afnd_manual (líneas 63-91) y su auxiliar _pedir_transiciones_afnd (líneas 94-139) reutilizan el patrón de entrada con while en las líneas 73-75 y 82-85. La condición de la línea 125, if (origen, simbolo) in delta, es la que distingue la Fase 2 de

la Fase 1: mientras en la primera fase un par repetido era un error fatal, ahora se le indica al usuario que debe unir los destinos con |. En la línea 131 se obtienen los destinos con `_separar_destinos(...)` y en la línea 132 se verifica que cada uno pertenezca a Q mediante `invalidos = [d for d in destinos if d not in Q]`.

La función `cargar_afnd_desde_archivo` (líneas 146-281) comparte el esqueleto de `cargador.cargar_afd_desde_archivo`, con tres diferencias puntuales. Entre las líneas 211 y 219 se maneja la línea opcional TIPO=: si su valor no es AFND, se agrega un aviso pero no se bloquea la carga. En la línea 194 se aplica el mismo cambio de criterio que en la versión manual ante un par (origen, símbolo) repetido. Finalmente, en la línea 201 se separan los destinos con `_separar_destinos(texto_destino)`, sin validar todavía que pertenezcan a Q; esa verificación se delega a `validador_afnd.py`, siguiendo el mismo diseño de `cargador.py`, que tampoco valida Q al leer el archivo.

validador.py

Este módulo tampoco tiene cambios de código. La función `validar_estructura` (líneas 29-80) verifica que `q0` pertenezca a Q, que F sea subconjunto de Q, que las transiciones sean consistentes y detecta las transiciones incompletas. La función `analizar_estructura` (líneas 83-116) calcula los estados alcanzables mediante un recorrido del grafo, y a partir de ellos obtiene los inaccesibles (Q - alcanzables) y los finales alcanzables (F & alcanzables). La función `completar_con_estado_trampa` (líneas 119-150) agrega el estado `q_trampa` junto con sus auto-transiciones.

La novedad en la Fase 2 es que `analizar_estructura` ya no se ejecuta únicamente dentro de la validación: `main.imprimir_analisis_estructural` (línea 347 de `main.py`) la invoca por separado para la opción 13, y también se llama automáticamente dentro de `accion_validar_afd` (línea 393). Se trata del mismo código con dos puntos de entrada, lo que ilustra la reutilización de funciones de la Fase 1.

validador_afnd.py

La función `validar_estructura_afnd` (líneas 15-46) sigue la misma lógica que `validador.validar_estructura`, pero es más breve. En la línea 24 se verifica que `afnd.q0` pertenezca a `afnd.Q`, y en la línea 28 se calcula `faltantes_F = afnd.F - afnd.Q`, igual que en el caso del AFD. La diferencia aparece en el ciclo sobre `afnd.delta.items()` (líneas 31-38): en la línea 36, `faltantes_destino = destinos - afnd.Q` emplea una resta de conjuntos, ya que `destinos` es un `frozenset` que puede contener varios estados, en lugar de comparar una sola cadena contra Q.

Es importante señalar que en este módulo no existe ningún cálculo de transiciones incompletas ni llamada para completar con un estado de trampa, porque un AFND no tiene ese concepto, tal como se desprende de `automata.AFND.transicion` (línea 121).

conversor.py

Este módulo es completamente nuevo en la Fase 2 y constituye el núcleo de la segunda funcionalidad del proyecto. En la línea 29 se define la constante `ALFABETO_ETIQUETAS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"`, que sirve de apoyo para nombrar los macroestados. La función `_siguiente_etiqueta` (líneas 32-43) convierte un índice numérico en una etiqueta al estilo de las columnas de Excel; la línea 41, `indice, resto = divmod(indice - 1, 26)`, es la que explica la secuencia A, B, ..., Z, AA, AB. La función `formatear_subconjunto` (líneas 46-51) es una acción de presentación que decide si un macroestado se imprime como $\emptyset$ o como $\{q_0, q_1\}$, y es utilizada tanto por este módulo como por `main.py` en las opciones 5, 8 y 9.

La función `construir_afd_desde_afnd` (líneas 54-110) implementa el algoritmo de construcción de subconjuntos. En la línea 67 se define el macroestado inicial como `frozenset({afnd.q0})`. Entre las líneas 69 y 72 se inicializan las variables del algoritmo: `etiquetas` (diccionario), `orden_descubrimiento` (lista), `pendientes` (una deque que funciona como cola) y `delta_afd` (diccionario que se va completando). La línea 74 (`while pendientes:`) junto con la línea 75 (`actual = pendientes.popleft()`) conforman un recorrido en anchura (BFS). Entre las líneas 76 y 79, dos ciclos anidados, uno por símbolo y otro por cada estado del macroestado actual, calculan la unión de destinos. Entre las líneas 81 y 84 se evalúa si el macroestado resultante es nuevo, en cuyo caso se le asigna una etiqueta y se encola. En la línea 86 se registra la transición correspondiente del AFD generado.

Una vez vaciada la cola, entre las líneas 88 y 93 se calculan `Q_afd`, formado por todas las etiquetas, y `F_afd`, formado por los macroestados que intersectan con `afnd.F` (línea 92, `if subconjunto & afnd.F`). Entre las líneas 95 y 105 se construye el AFD de salida y se marca `validado = True` (línea 105), ya que por construcción el resultado siempre es completo y determinista. Finalmente, entre las líneas 107 y 109 se arma la `tabla_equivalencias`.

simulador.py

Este módulo no presenta cambios de código. Las funciones `evaluar_cadena` (líneas 54-94), `evaluar_lote` (líneas 97-121) e `imprimir_traza` (líneas 124-147) conservan la misma lógica de la Fase 1.

En la Fase 2, estas funciones reciben el AFD que devuelve `main.obtener_afd_operable(...)` (línea 109 de `main.py`), el cual puede ser un AFD cargado directamente o el AFD generado a partir de una conversión. El simulador no necesita saber de dónde proviene el autómata, lo que demuestra que reutilizarlo sin modificaciones cumple con el principio de compatibilidad establecido en el enunciado.

colores.py

Este módulo no tiene cambios y es un utilitario de presentación basado en códigos ANSI. No aporta acciones, datos ni restricciones propias del dominio de autómatas; su relevancia se limita a las decisiones de diseño, particularmente la elección de implementarlo manualmente en lugar de depender de una biblioteca externa como colorama.

main.py

Este archivo fue reescrito para la Fase 2 y concentra la mayor parte de las decisiones de diseño. Las funciones `mostrar_menu` (líneas 57-76) y `pedir_opcion` (líneas 79-93) reciben como dato de entrada la opción del usuario y exigen que sea un entero entre 1 y 15, lo cual se verifica con `texto.isdigit()` (línea 86) y `1 <= opcion <= 15` (línea 90). La función `hay_automata_cargado` (líneas 100-106) es una guardia simple que utilizan la opción 5 y el primer paso de la opción 6.

La función `obtener_afd_operable` (líneas 109-136) es la pieza central de la compatibilidad entre ambas fases. En la línea 126, la condición `isinstance(automata_cargado, AFD)` determina si el autómata se devuelve tal cual o si, por tratarse de un AFND, se exige que exista `afd_generado` (línea 133). Esta función es utilizada por las opciones 10, 11 y 13.

Las estructuras `RegistroHistorial` (líneas 144-150), una `dataclass`, e `Historial` (líneas 153-189), su clase contenedora, gestionan el registro de evaluaciones; el método `mostrar` (líneas 176-181) implementa la opción 12. Las funciones `accion_crear_manual`, `accion_cargar_archivo`, `accion_crear_manual_afnd` y `accion_cargar_archivo_afnd` (líneas 196-239) son envoltorios delgados: la lógica real reside en `cargador.py` y `cargador_afnd.py`, y aquí solo se añaden los mensajes de éxito o error.

Las funciones `accion_mostrar_definicion` y `accion_mostrar_tabla` (líneas 246-282), junto con sus equivalentes para AFND (líneas 285-329), calculan dinámicamente los anchos `ancho_estado` y `ancho_celda` (líneas 264 y 310-311) según el contenido más largo, de manera que la tabla se mantenga alineada tanto para AFD como para AFND. La función `accion_mostrar_automata_cargado` (líneas 332-340) es el primer despachador basado en `isinstance` (línea 335), patrón que se repite en `accion_validar_automata_cargado` (líneas 413-419, línea 416), la cual delega en `accion_validar_afd` (líneas 363-393) o en `accion_validar_afnd` (líneas 396-410). La función `imprimir_analisis_estructural` (líneas 347-360) es una acción compartida, invocada automáticamente por la opción 6 y bajo demanda por la opción 13.

La función `accion_convertir` (líneas 426-442) aplica dos guardias en cascada antes de llamar a `conversor.construir_afd_desde_afnd` (línea 439): primero verifica que el autómata sea un AFND (línea 432) y luego que esté validado (línea 435). La función

accion_mostrar_equivalencias (líneas 445-454) presenta la tabla_equivalencias. Las funciones accion_evaluar_cadena (líneas 461-467) y accion_evaluar_archivo (líneas 470-493) mantienen la lógica de la Fase 1, con la única diferencia de que reciben un AFD ya resuelto por obtener_afd_operable.

Por último, la función main() (líneas 500-603) declara las variables de sesión (líneas 512-515) y contiene la estructura condicional de 15 ramas (líneas 526-603). En las opciones 1 a 4 (líneas 527-548) se repite un mismo patrón: cada una reinicia afd_generado, tabla_equivalencias e historial, lo que implementa en código la regla de la Fase 2 según la cual, al cargar un nuevo autómatas, se limpian los datos anteriores.

Restricciones

Las reglas de validación de los (AFD vs AFND) son las reglas establecidas en la Fase 1 del proyecto y las restricciones pedidas para el proyecto

Restricciones generales del proyecto

- Debe funcionar con cualquier AFD/AFND válido, no solo con un caso particular.
- Prohibido usar librerías que implementen autómatas o la construcción de subconjuntos.
- ð debe ser un diccionario genérico, nunca una cadena de if/elif por autómatas específico (esto lo puedes respaldar señalando automata.py:59-65 y automata.py:113-121).
- Lectura y validación sintáctica de archivos mediante expresiones regulares.
- Separación de responsabilidades entre carga, validación, conversión y simulación (uno por módulo: cargador*.py, validador*.py, conversor.py, simulador.py).
- Las funciones nuevas se integran al programa de Fase 1, no son una solución aparte.

Reglas de acceso entre opciones del menú

- No se puede convertir (opción 7) sin un AFND válido cargado.
- No se puede evaluar cadenas ni hacer análisis estructural (10, 11, 13) sobre un AFND sin convertir antes.
- Un AFD cargado directamente (opciones 1/2) nunca necesita conversión — puede evaluarse de inmediato tras validarse.

Diagramas de flujo del proyecto

Diagrama principal

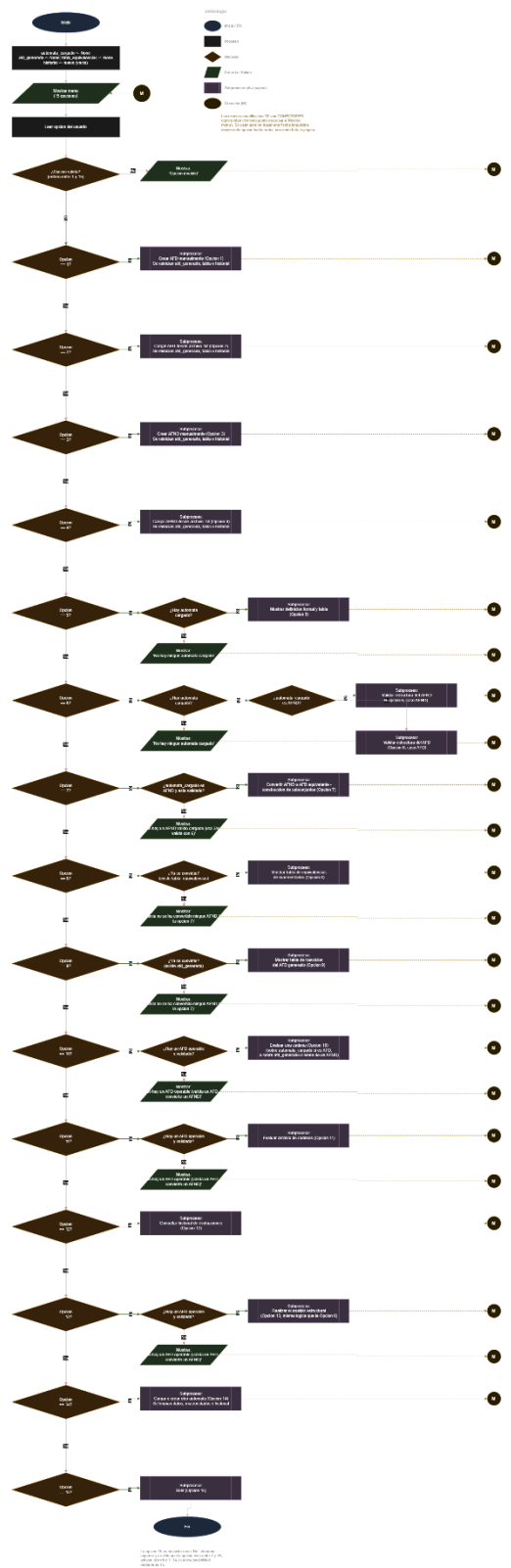


Diagrama Crear un AFD manualmente

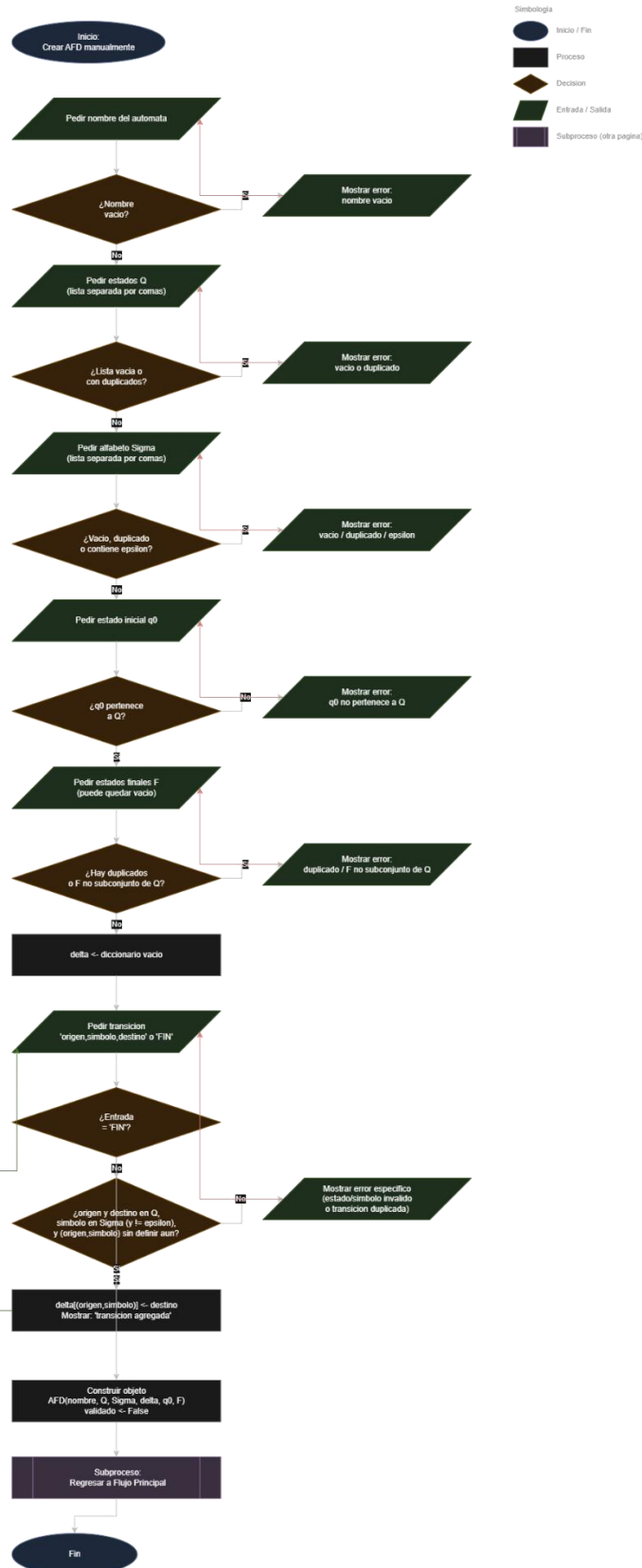


Diagrama Cargar un AFD desde archivo .txt



Diagrama Crear un AFND manualmente

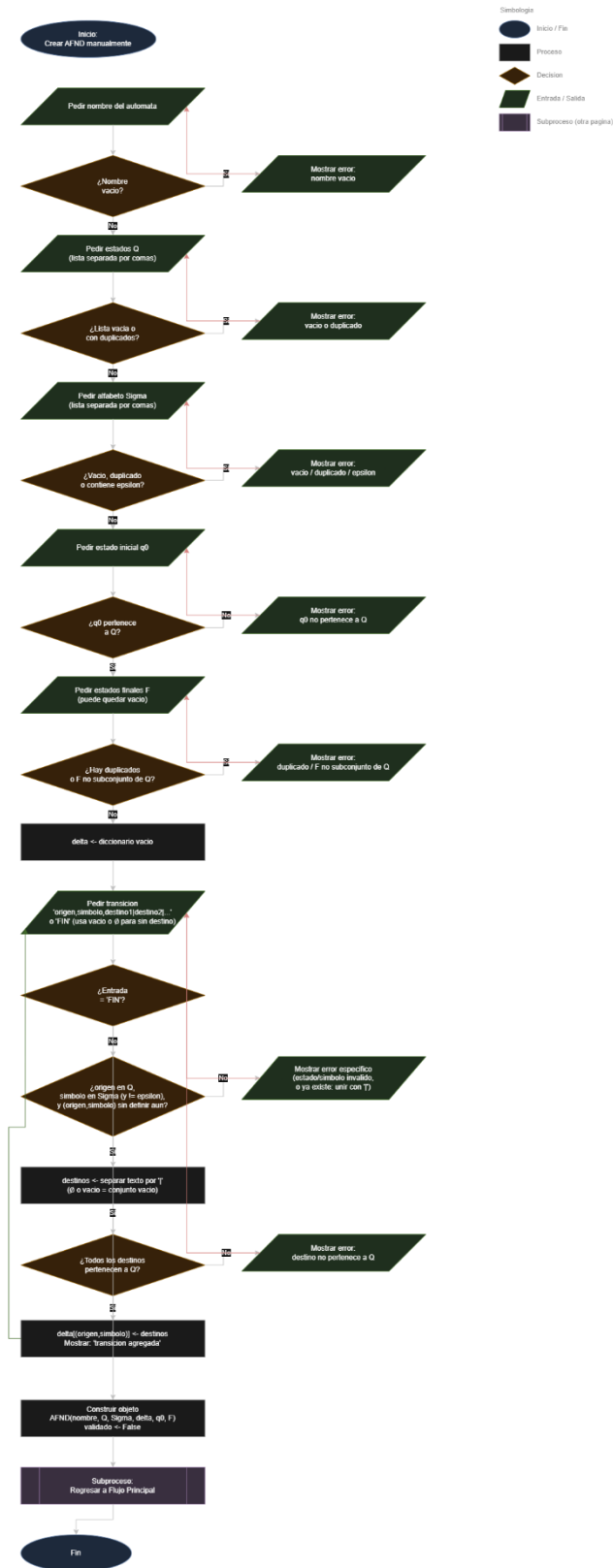


Diagrama Cargar un AFND desde archivo .txt

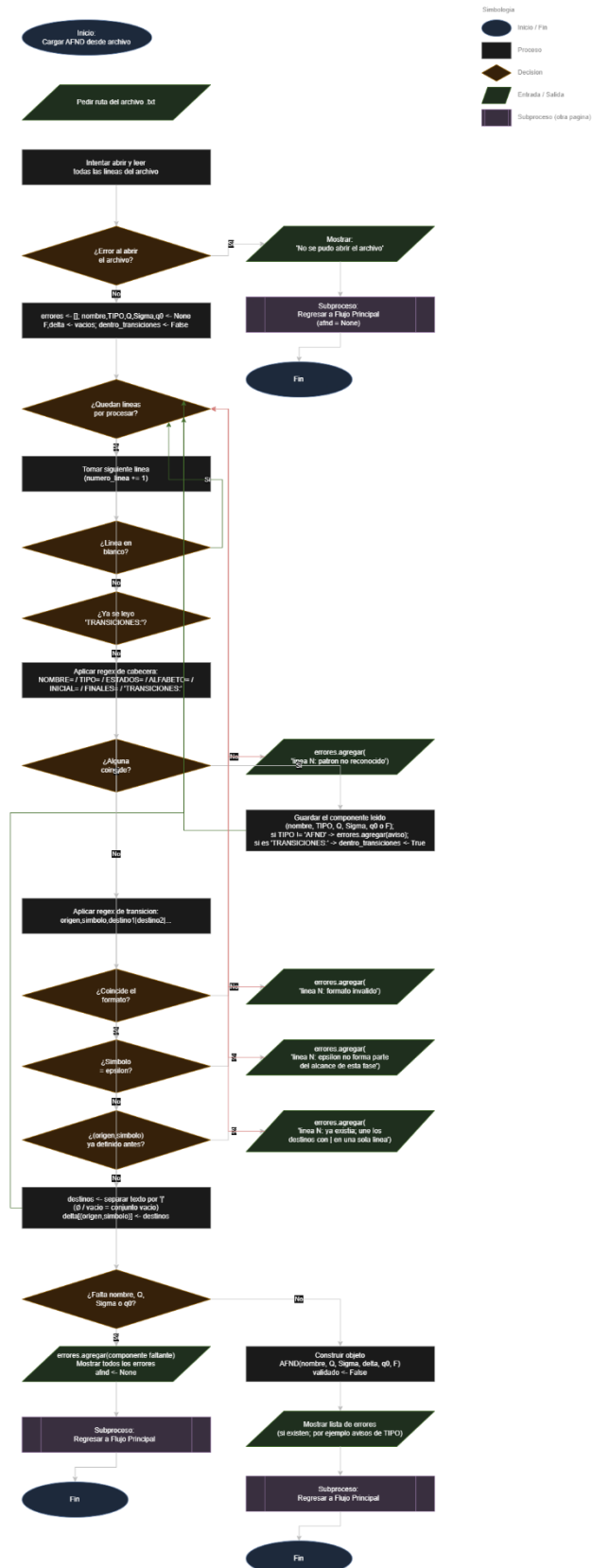


Diagrama Mostrar la definición formal y la tabla del autómata cargado

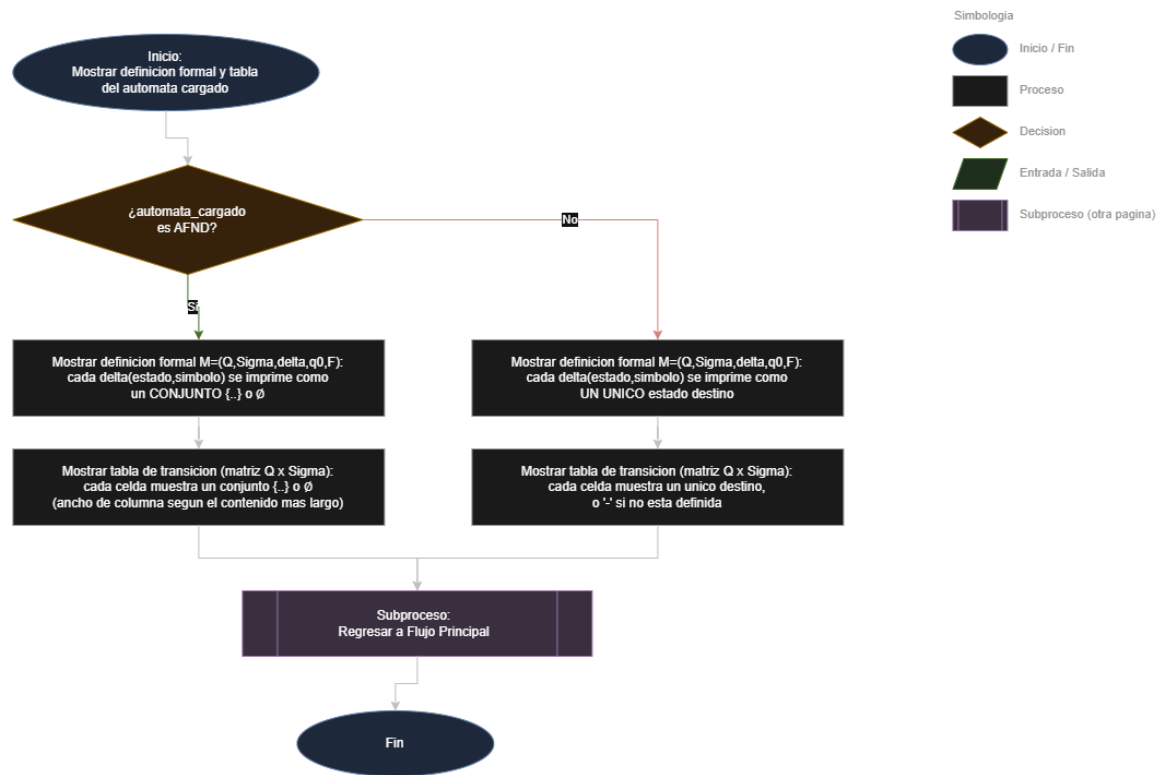
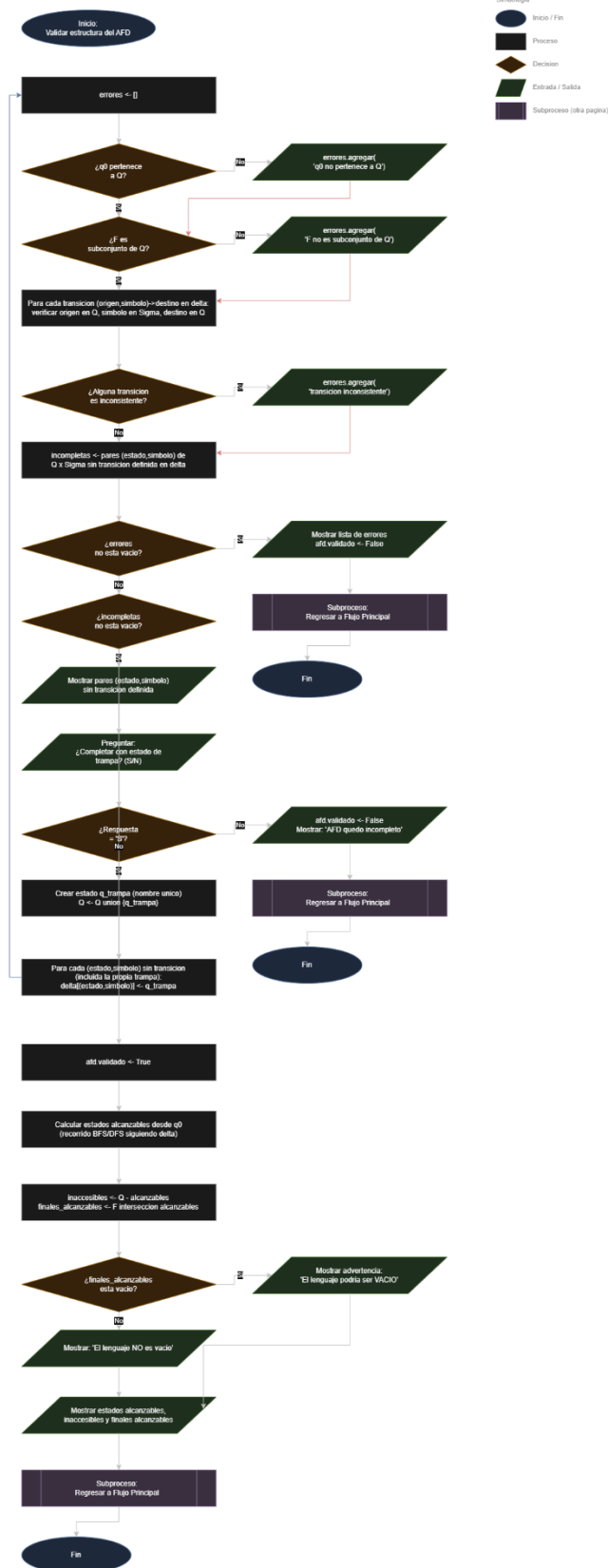
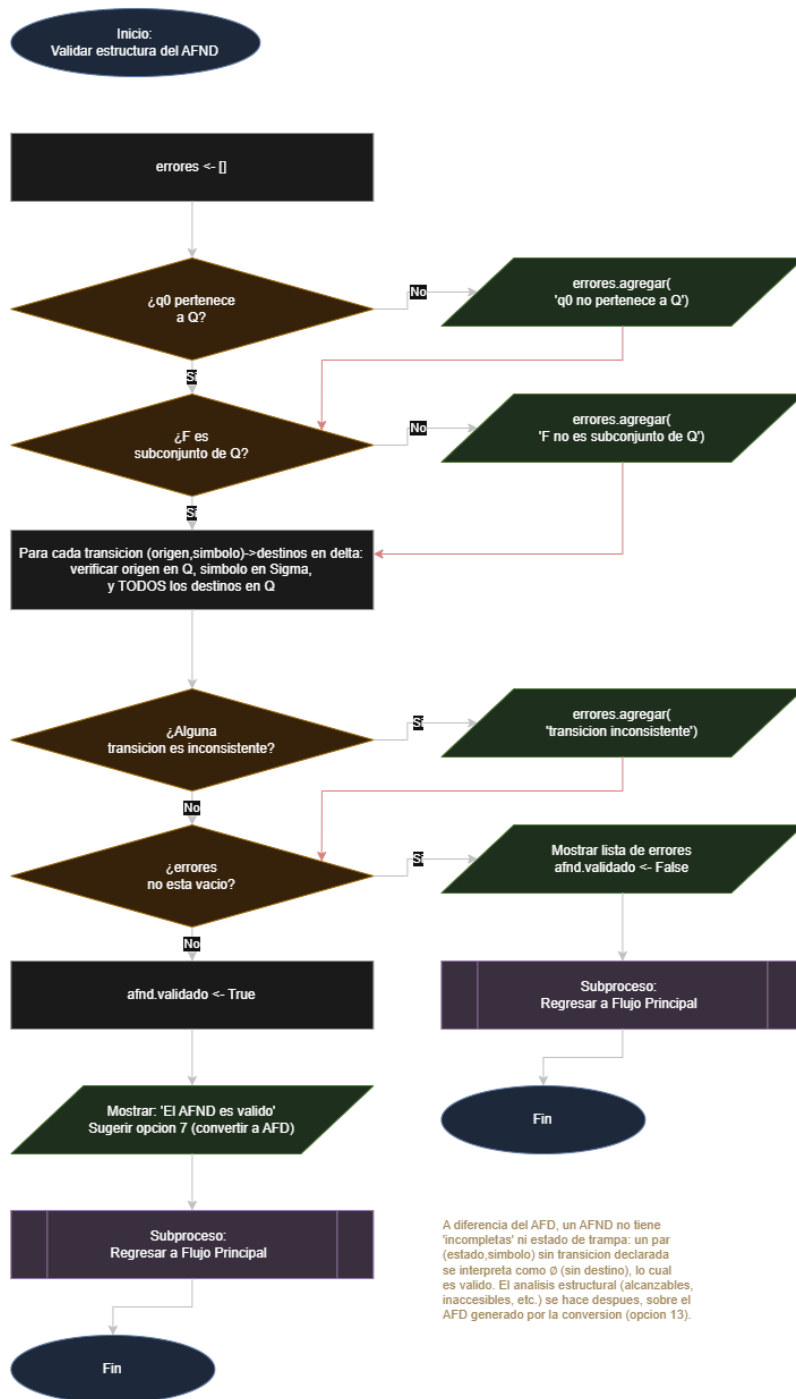


Diagrama Validar la estructura del autómata





A diferencia del AFD, un AFND no tiene 'incompletas' ni estado de trampa: un par (estado,simbolo) sin transicion declarada se interpreta como $\emptyset$ (sin destino), lo cual es valido. El analisis estructural (alcanzables, inaccesibles, etc.) se hace despues, sobre el AFD generado por la conversion (opcion 13).

Diagrama Convertir el AFND cargado en un AFD equivalente

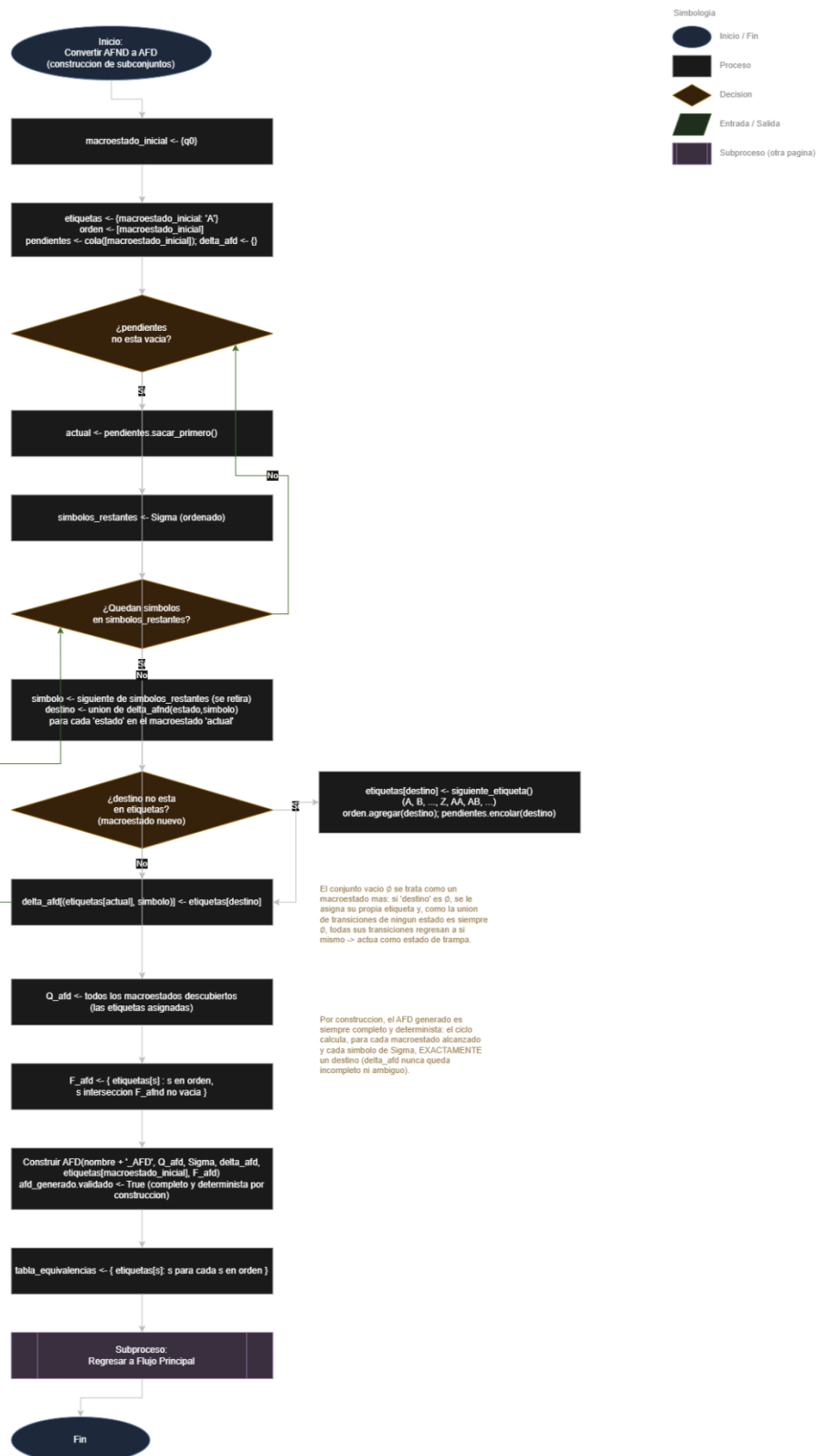


Diagrama Mostrar la tabla de equivalencias de macroestados

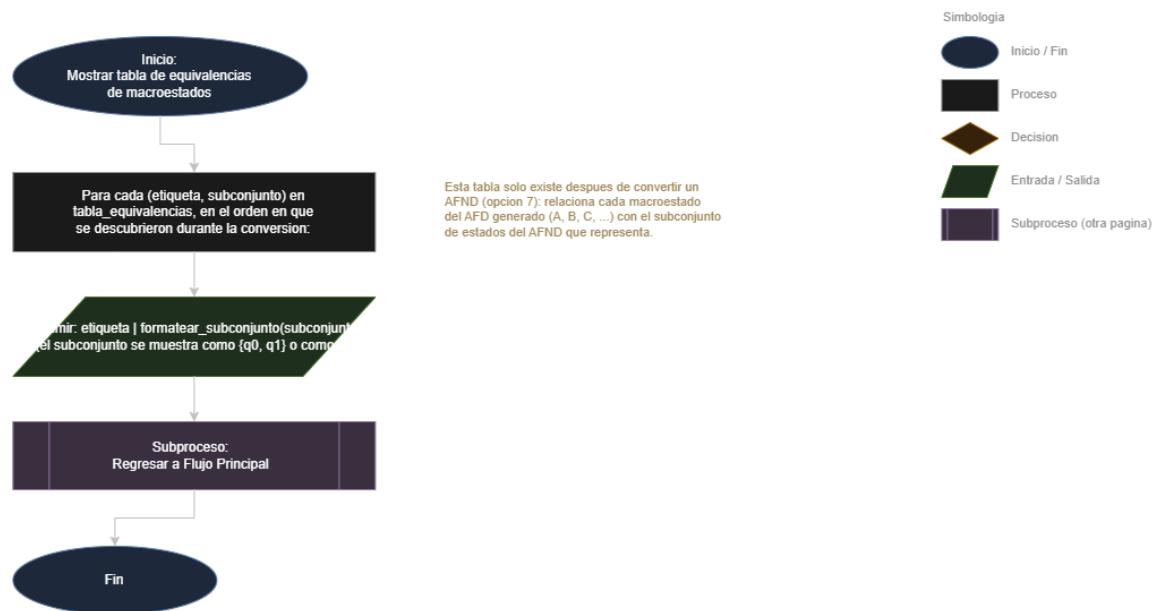


Diagrama Mostrar la tabla de transición del AFD generado

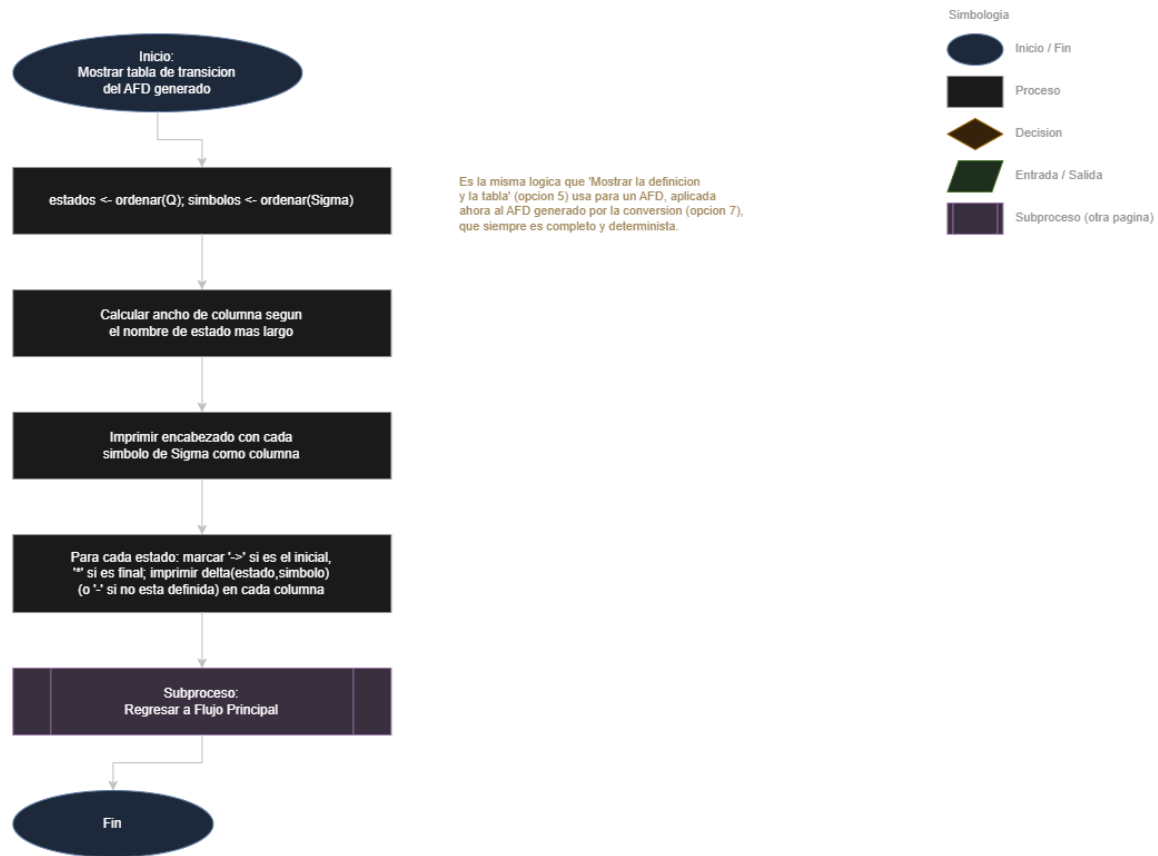


Diagrama Evaluar una cadena

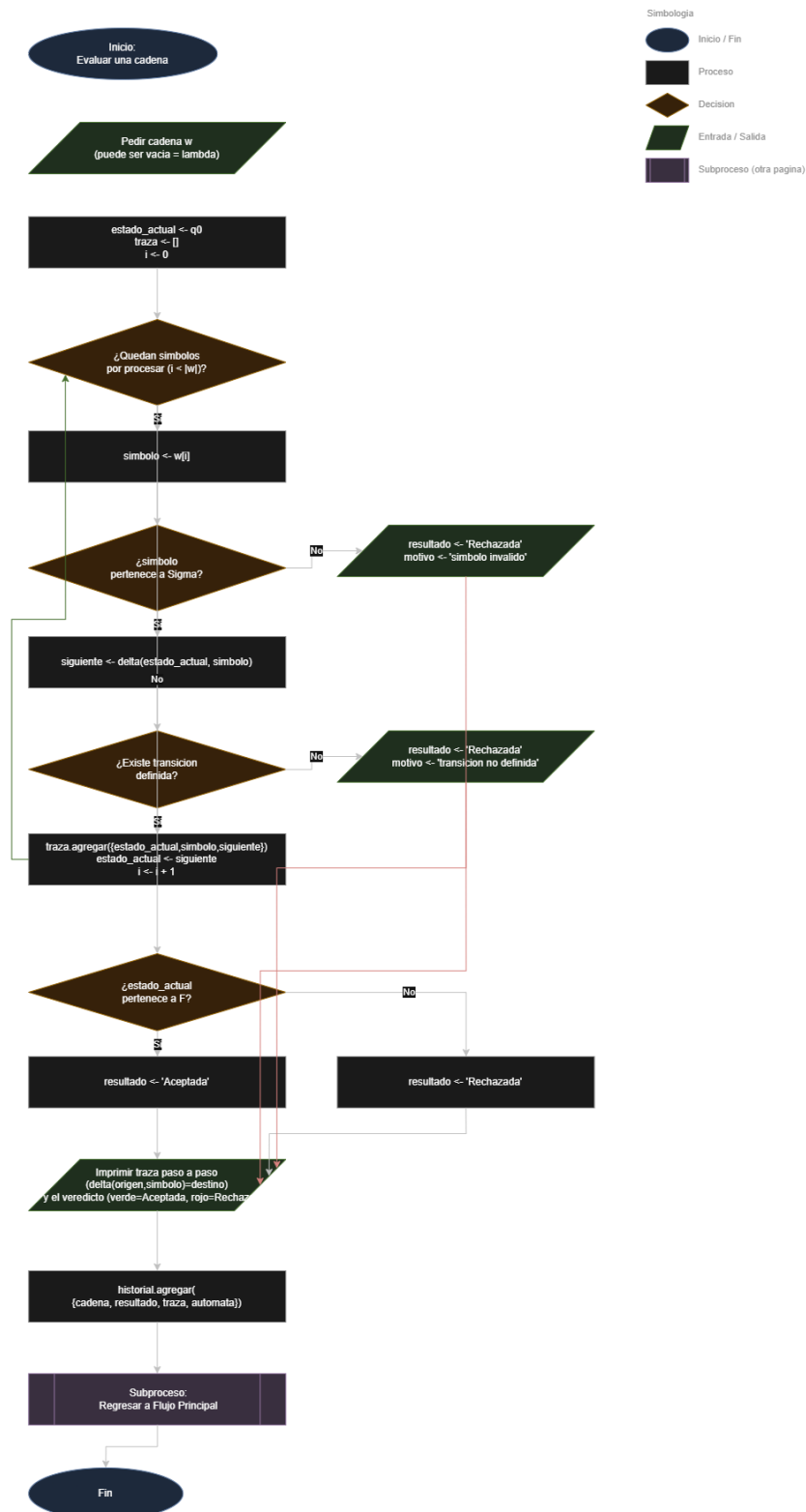


Diagrama Evaluar un archivo de cadenas

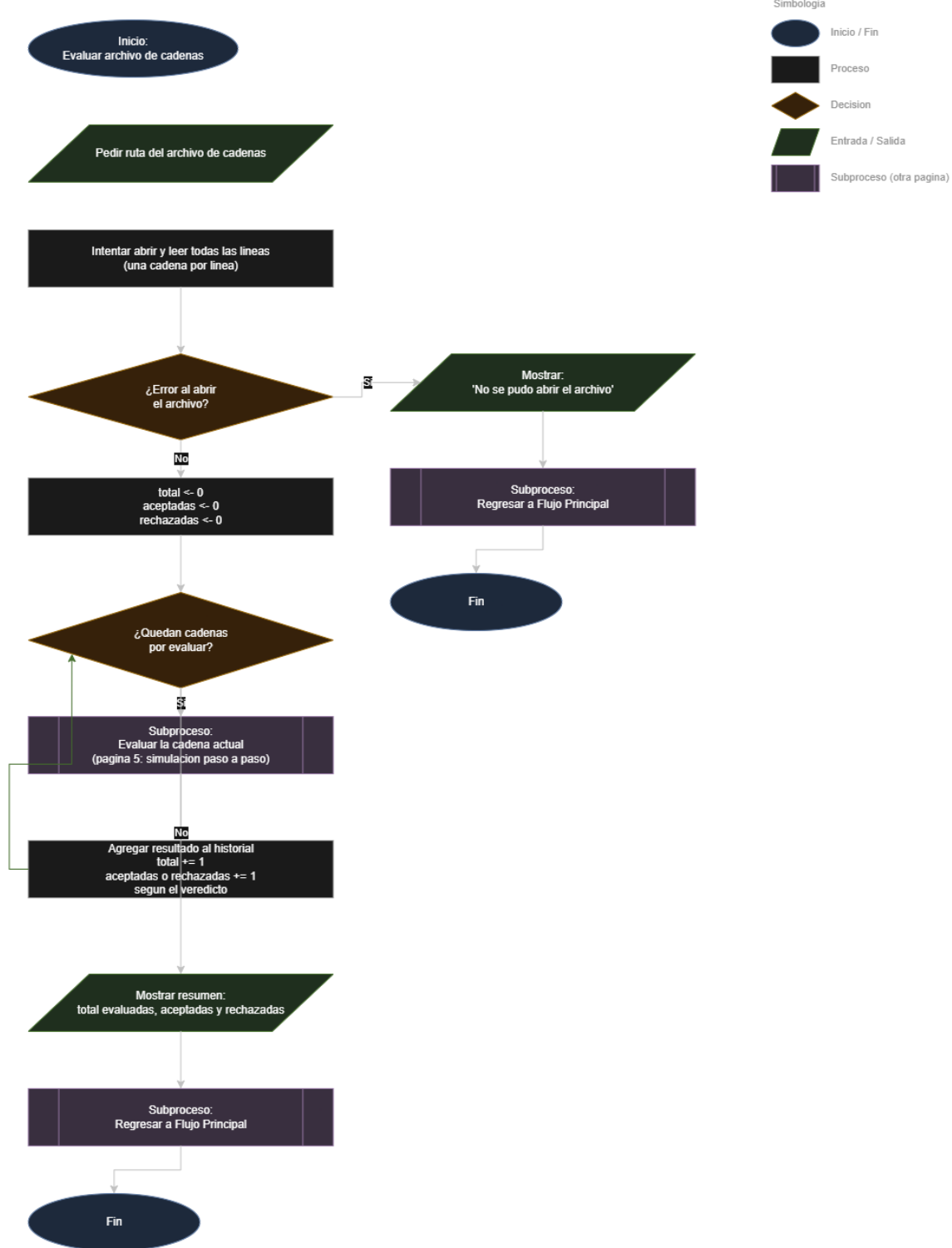


Diagrama Consultar el historial de evaluaciones

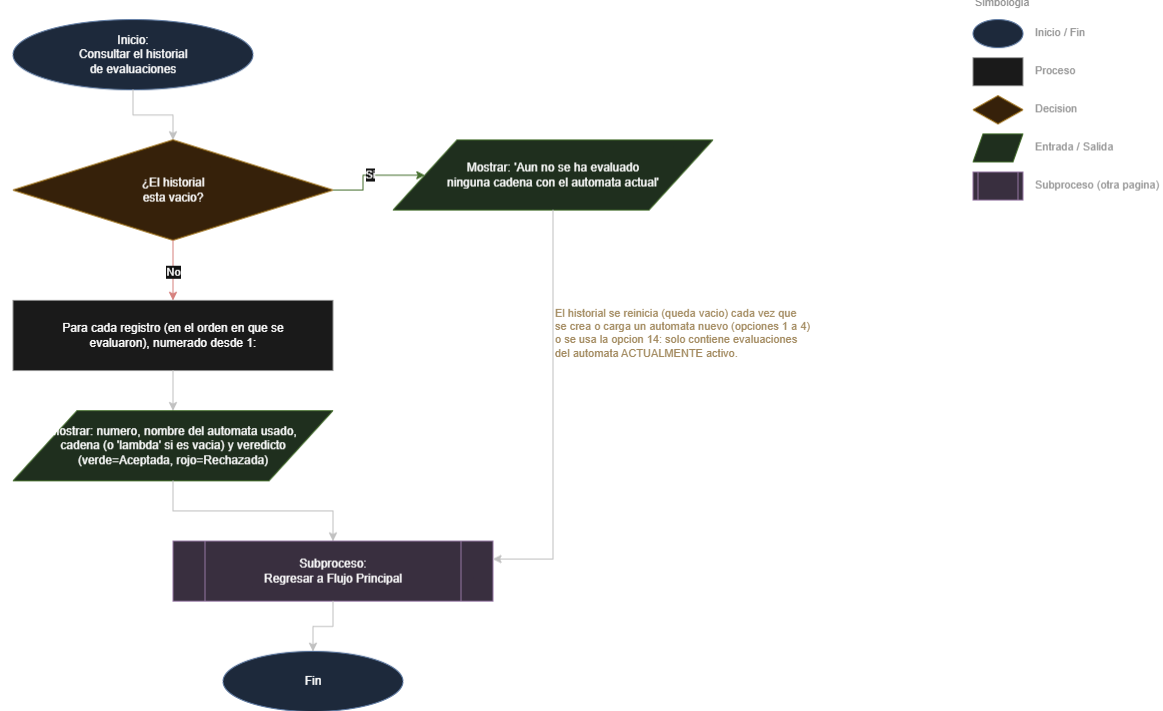


Diagrama Realizar el análisis estructural

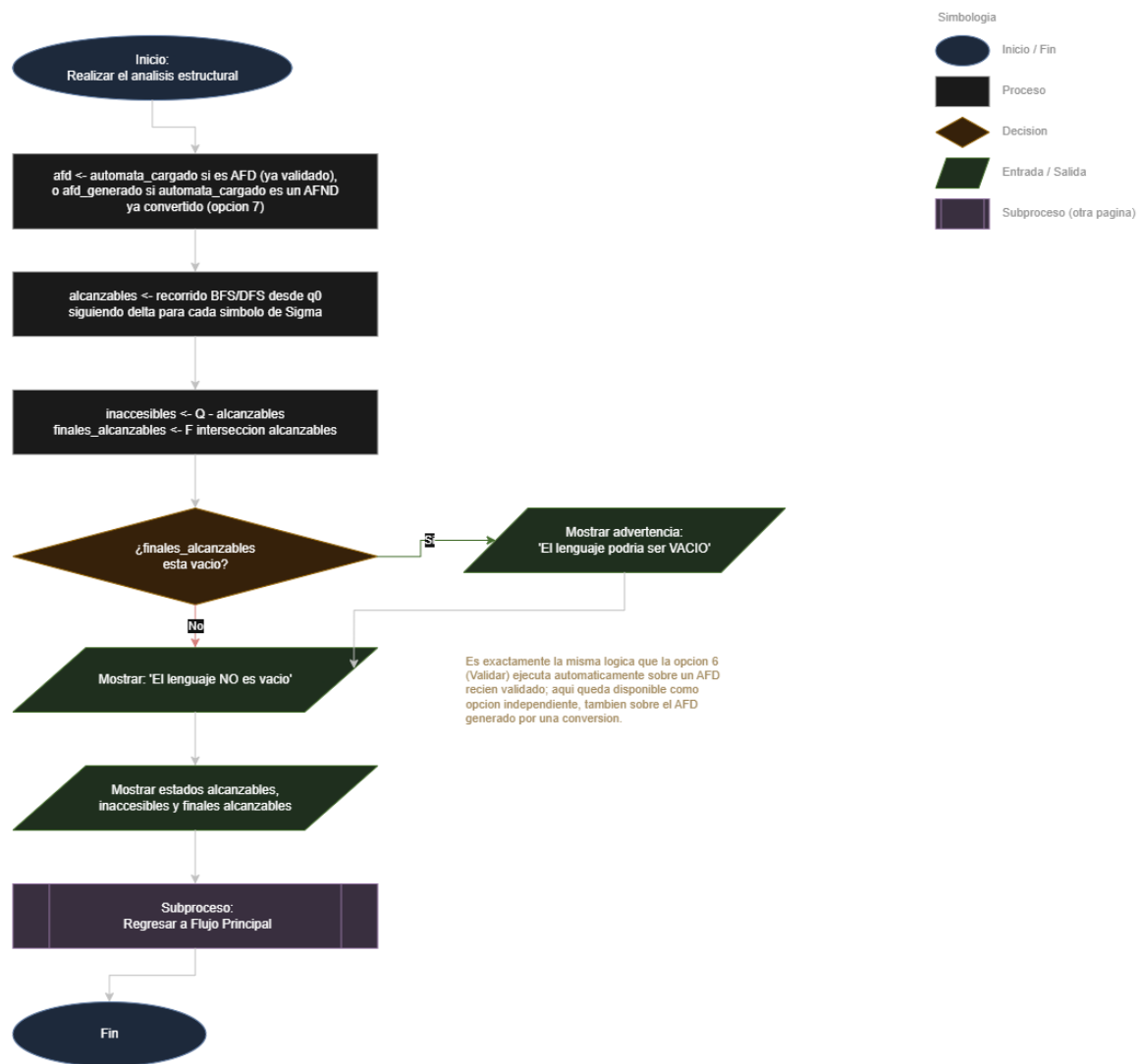


Diagrama Cargar o crear otro autómata

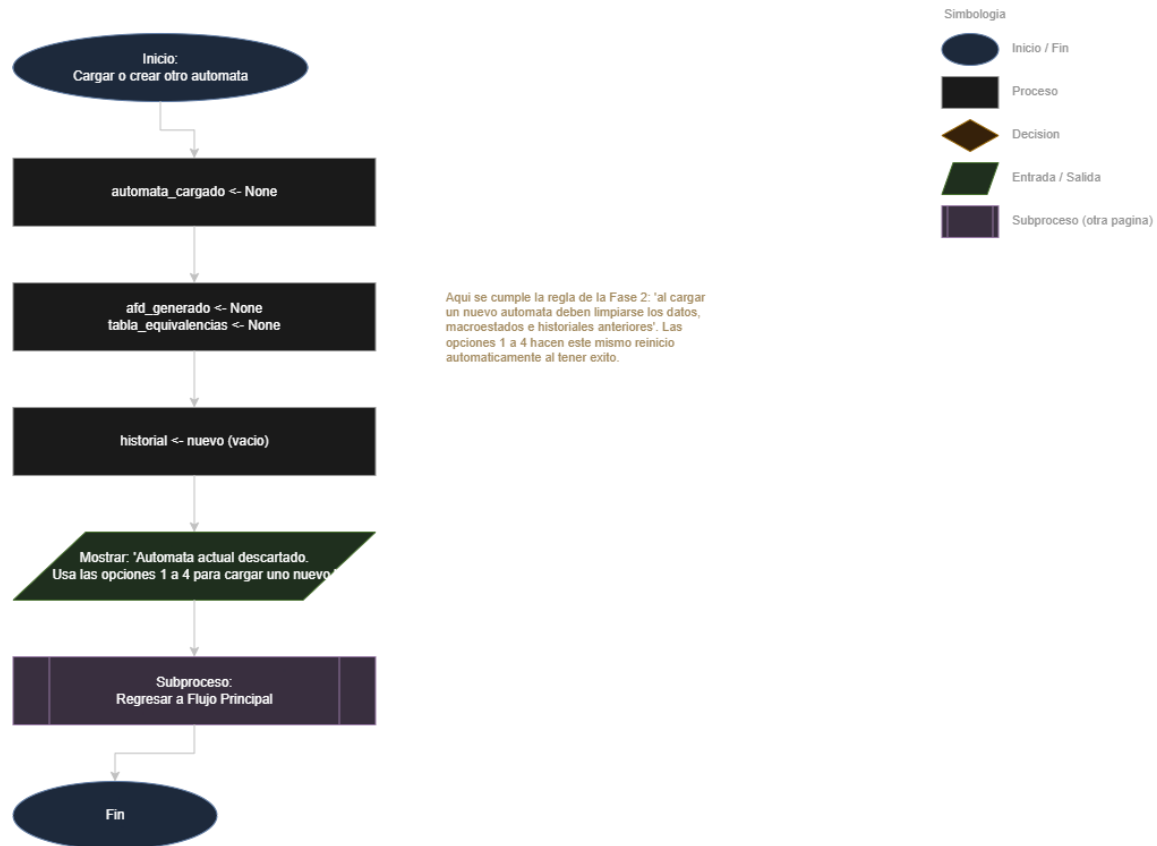


Diagrama Salir



Casos de prueba

En el caso de los casos de prueba se adjuntarán en la entrega algunos de los archivos TXT que se pedían en las pruebas para poder hacerlo de manera más realista y que no exista duda de lo que puede hacer el programa se encuentra en la carpeta de ejemplos sus mismos nombres demuestran los errores que tratan de ejemplificar o tienen un nombre relacionado al contexto de lo que tratan de verificar se adjuntaron 11 .txt